

Week10

☰ 연습과제

06 차원 축소

차원 축소(Dimension Reduction) 개요

차원 축소의 필요성

- 차원이 증가할수록 데이터 포인트 간 거리가 기하급수적으로 멀어지고 희소(sparse)한 구조가 됨
- 수백 개 이상의 피쳐 → 예측 신뢰도 ↓, 피쳐 간 상관관계 ↑
- 선형 회귀 등 선형 모델 → 입력 변수 간 상관관계가 높을 경우 **다중 공선성 문제**로 예측 성능 저하

차원 축소의 두 가지 방식

구분	설명
피쳐 선택(feature selection)	불필요한 피쳐는 제거하고 데이터 특징을 잘 나타내는 주요 피쳐만 선택
피쳐 추출(feature extraction)	기존 피쳐를 저차원의 중요 피쳐로 압축 → 기존 피쳐와 완전히 다른 값

차원 축소의 진짜 의미

- 단순 압축이 아니라 **잠재적인 요소(Latent Factor)**를 추출하는 데 핵심 의미가 있음
- 활용 분야
 - 이미지 → 잠재 특성을 추출해 과적합(overfitting) 영향력 감소 → 예측 성능 향상
 - 텍스트 문서 → 단어 구성에서 숨겨진 시맨틱(Semantic)/토픽(Topic) 추출 (SVD, NMF가 기반 알고리즘)

PCA(Principal Component Analysis)

PCA 개요

정의

- 가장 대표적인 차원 축소 기법
- 여러 변수 간 상관관계를 이용해 이를 대표하는 **주성분(Principal Component)** 을 추출해 차원을 축소
- 기존 데이터의 정보 유실을 최소화하기 위해 → **가장 높은 분산을 가지는 데이터의 축을** 찾음
 - 분산이 데이터의 특성을 가장 잘 나타낸다고 간주

PCA 축 생성 방식

- 1번째 축 → 가장 큰 데이터 변동성(Variance) 방향
- 2번째 축 → 1번째 축에 직각이 되는 벡터(직교 벡터)
- 3번째 축 → 다시 2번째 축과 직각이 되는 벡터
- → 생성된 벡터 축 개수만큼의 차원으로 축소

PCA의 선형대수 관점

핵심 개념

- 입력 데이터의 **공분산 행렬(Covariance Matrix)** 을 **고유값 분해**
- 고유벡터에 입력 데이터를 선형 변환 → 차원 축소
- 고유벡터: PCA의 주성분 벡터 → 입력 데이터의 분산이 큰 방향
- 고유값(eigenvalue): 고유벡터의 크기 = 입력 데이터의 분산

분산 vs 공분산

- 분산 → 한 개 변수의 데이터 변동
- 공분산 → 두 변수 간의 변동 (예: $\text{Cov}(X, Y) > 0 \rightarrow X$ 증가할 때 Y 도 증가)
- 공분산 행렬 → 여러 변수의 공분산을 포함하는 정방형 행렬

공분산 행렬의 특성

- 정방행렬(Square Matrix)이며 **대칭행렬(Symmetric Matrix)** ($A^T = A$)

- 대칭행렬 → 항상 고유벡터를 직교행렬(orthogonal matrix)로, 고유값을 정방 행렬로 대각화 가능
- 분해 식: $C = P \Sigma P^T$
 - P : $n \times n$ 직교행렬
 - Σ : $n \times n$ 정방행렬
 - P^T : P 의 전치 행렬

PCA 수행 스텝

1. 입력 데이터 세트의 공분산 행렬 생성
2. 공분산 행렬의 고유벡터와 고유값 계산
3. 고유값이 큰 순으로 K개(PCA 변환 차수)만큼 고유벡터 추출
4. 추출된 고유벡터를 이용해 입력 데이터를 변환

사이킷런 PCA 실습 - 붓꽃 데이터

실습 흐름

원본 4차원 데이터 → 스케일링 → 2개 PCA 컴포넌트로 축소 → 시각화 비교

핵심 포인트

- **스케일링 필수**: PCA는 여러 속성의 값을 연산하므로 속성 스케일의 영향을 받음
 - StandardScaler로 평균 0, 분산 1인 표준 정규 분포로 변환
- **PCA 클래스**: `sklearn.decomposition.PCA`
 - 생성 파라미터 `n_components` → 변환할 차원 수
 - `fit(입력 데이터)` → `transform(입력 데이터)` 호출
- **explained_variance_ratio_**: 전체 변동성에서 개별 PCA 컴포넌트별로 차지하는 변동성 비율
 - 붓꽃 결과: [0.7296, 0.2285] → 2개 컴포넌트만으로 약 95% 설명 가능

PCA 변환 후 분류 성능 비교 (붓꽃)

구분	평균 정확도
원본 데이터 (4차원)	0.96

구분	평균 정확도
PCA 변환 데이터 (2차원)	0.88

→ 8% 정확도 하락이지만 속성 개수가 50% 감소한 점을 고려하면 PCA 변환 후에도 원본 특성을 상당 부분 유지

사이킷런 PCA 실습 - 신용카드 데이터

데이터 개요

- UCI Credit Card Clients Data Set (30,000 레코드 × 24 속성)
- Target: `default payment next month` (다음달 연체 여부, 연체 1 / 정상 0)

전처리

- 칼럼명 변경: `PAY_0` → `PAY_1`, `default payment next month` → `default`
- Target 분리: `y_target = df['default'], X_features = df.drop('default', axis=1)`

속성 상관관계 분석

- `BILL_AMT1` ~ `BILL_AMT6`의 6개 속성끼리 상관도 대부분 0.9 이상 (매우 높음)
- `PAY_1` ~ `PAY_6`도 상관도 높음
- → 높은 상관도 = 소수의 PCA만으로 변동성 수용 가능

`BILL_AMT1~6` → 2개 PCA 결과

- `explained_variance_ratio`: [0.9056, 0.0510]
- 단 2개 PCA 컴포넌트만으로 약 95% 이상 설명 가능 (첫 번째 축이 90% 수용)

분류 예측 성능 비교 (랜덤 포레스트, CV=3)

구분	평균 정확도
원본 데이터 (23개 속성)	0.8170
PCA 변환 데이터 (6개 컴포넌트)	0.7973

→ 약 1~2% 성능 저하만 발생 → 전체 속성의 1/4 만으로도 성능 유지 가능 = PCA의 뛰어난 압축 능력

→ 컴퓨터 비전 분야에서 활발히 사용 (Eigen-face 등)

LDA(Linear Discriminant Analysis)

LDA 개요

정의

- 선형 판별 분석법
- PCA와 유사하게 저차원 공간에 투영해 차원 축소
- **PCA와의 결정적 차이점:** LDA는 지도학습의 분류(Classification)에서 사용하기 쉽도록 **개별 클래스를 분별할 수 있는 기준을 최대한 유지**하면서 차원 축소

PCA vs LDA 핵심 비교

구분	PCA	LDA
학습 유형	비지도학습	지도학습 (클래스 결정값 필요)
축 선정 기준	데이터 변동성이 가장 큰 축	클래스 분리를 최대화하는 축
행렬 분해 대상	공분산 행렬	클래스 간/내부 분산 행렬

LDA의 축 찾기 방식

- 클래스 간 분산(between-class scatter)은 **최대한 크게**
- 클래스 내부 분산(within-class scatter)은 **최대한 작게**
- → 두 분산의 비율을 최대화하는 방식으로 차원 축소

LDA 수행 스텝

1. 클래스 내부(S_W)와 클래스 간(S_B) 분산 행렬을 구함 (피처의 평균 벡터 기반)
2. $(S_W)^T \times S_B$ 행렬을 고유벡터로 분해
3. 고유값이 큰 순으로 K개(LDA 변환 차수) 추출
4. 추출된 고유벡터로 새롭게 입력 데이터 변환

붓꽃 데이터에 LDA 적용

핵심 코드 흐름

- 클래스: `sklearn.discriminant_analysis.LinearDiscriminantAnalysis`

- 주의: `lda.fit(iris_scaled, iris.target)` → **결정값(target)**이 입력되어야 함 (지도학습)
- 변환 후 시각화 → PCA와 좌우 대칭 형태로 매우 유사한 결과

SVD(Singular Value Decomposition)

SVD 개요

정의

- 특이값 분해
- PCA와 유사한 행렬 분해 기법
- **PCA와의 차이점:** PCA는 정방행렬만 분해 가능, SVD는 행과 열 크기가 다른 행렬에도 적용 가능

SVD 분해 식

$$A = U \Sigma V^T$$

행렬	차원	의미
A	$m \times n$	원본 행렬
U	$m \times m$	좌측 특이벡터 (직교 벡터)
Σ	$m \times n$	대각행렬, 대각 원소 = 특이값(singular value)
V^T	$n \times n$	우측 특이벡터의 전치 (직교 벡터)

Compact SVD vs Truncated SVD

구분	설명
Compact SVD	Σ 에서 특이값이 0인 부분 제거 → U는 $m \times p$, Σ 는 $p \times p$, V^T 는 $p \times n$
Truncated SVD	Σ 의 대각원소 중 상위 일부만 추출 → 더 작은 차원으로 분해, 근사적 복원

넘파이 SVD 실습

기본 흐름

- 모듈: `from numpy.linalg import svd`
- 호출: `U, Sigma, Vt = svd(a)`

- $U \rightarrow (m, m)$, $\Sigma \rightarrow 0$ 이 아닌 값만 1차원으로 반환, $V^T \rightarrow (n, n)$
- 원본 복원: `np.dot(np.dot(U, Sigma_mat), Vt)` (Σ 를 대각행렬로 변환 필요)

데이터 의존성 → Σ 변화

- 행렬 로우 간 의존성을 부여하면 → Σ 값 중 일부가 0이 됨
- 0이 아닌 Σ 의 개수 = 행렬의 **랭크(Rank)** = 선형 독립인 로우 벡터 개수
- Σ 0에 대응하는 U , V^T 데이터를 제외하고도 원본 복원 가능

Truncated SVD 실습 (사이파이)

특징

- 넘파이는 SVD만 지원, 사이파이는 SVD + **Truncated SVD** 모두 지원
- Truncated SVD는 희소 행렬에만 지원 → `scipy.sparse.linalg.svds` 사용
- Σ 에서 상위 일부 특이값만 추출 → 원본을 완벽히 복원하지 못하지만 **근사적으로 복원**

사이킷런 TruncatedSVD 클래스

특징

- `sklearn.decomposition.TruncatedSVD`
- `fit()` → `transform()` 호출 (PCA와 유사)
- 사이파이의 svds와 달리 U , Σ , V^T 를 반환하지 않고 → **$U \times \Sigma$ 행렬에 선형 변환된 결과를 반환**

PCA vs TruncatedSVD

- 데이터를 스케일링하면 → 사이킷런의 SVD와 PCA는 **동일한 변환 수행**
- → PCA가 내부적으로 SVD 알고리즘으로 구현됨을 의미
- 차이점: PCA는 **밀집 행렬(Dense Matrix)** 만 변환 가능, SVD는 **희소 행렬(Sparse Matrix)** 도 변환 가능

SVD 활용 분야

- 컴퓨터 비전 → 이미지 압축, 패턴 인식, 신호 처리
- 텍스트의 토픽 모델링 기법인 **LSA(Latent Semantic Analysis)** 의 기반 알고리즘

NMF(Non-Negative Matrix Factorization)

NMF 개요

정의

- Truncated SVD와 같이 낮은 랭크를 통한 행렬 근사(Low-Rank Approximation) 방식의 변형
- 원본 행렬 내 모든 원소가 양수(0 이상) 일 때 → 두 개의 양수 행렬로 분해 가능

NMF 분해 식

$$V \approx W \times H$$

행렬	차원 예시	의미
V	4 × 6	원본 행렬
W	4 × 2	길고 가는 행렬 (원본의 행 크기 = W의 행 크기) → 잠재 요소의 값이 얼마나 되는지 대응
H	2 × 6	작고 넓은 행렬 (원본의 열 크기 = H의 열 크기) → 잠재 요소가 원본 열로 어떻게 구성됐는지 표현

→ 분해된 행렬은 잠재 요소(Latent Factor) 를 특성으로 가짐

사이킷런 NMF 실습

사용법

- `sklearn.decomposition.NMF`
- `nmf = NMF(n_components=2) → fit() → transform()`

NMF 활용 분야

- 이미지 변환·압축, 텍스트 토픽 모델링, 문서 유사도/클러스터링
- 추천 시스템(Recommendations) → 영화 추천 등
 - 사용자-Rating 데이터를 행렬 분해 → 평가하지 않은 상품의 잠재 요소 추출 → 평점 예측
 - = 잠재 요소(Latent Factoring) 기반 추천 방식